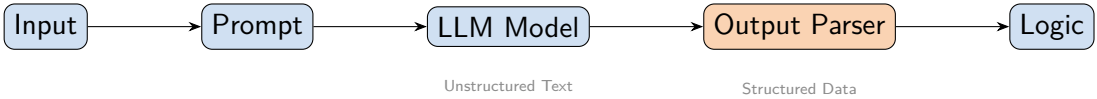
Output Parsing and Structured Output

LangChain Section 02

March 2026 — LangChain v1.2.x

The LangChain Pipeline



Output parsing happens after the model generates text.

The Fundamental Problem

LLMs return unstructured text. Applications need structured, typed data.

```
response = llm.invoke("Extract name and age from: John is 25")
# response = "The name is John and the age is 25 years old."
# How do you get name="John" and age=25 from this string?
```

Without structured output:

- Manually parsing strings is fragile
- No type safety or validation
- Hard to integrate with downstream systems

With structured output:

- Guaranteed schema compliance
- Type-safe data for your application
- Easier integration and testing

Output Parsing Strategies

Strategy	Pros	Cons
Prompting	Works everywhere	Unreliable
Function Calling	Very reliable	Requires support
JSON Mode	Good balance	Extra tokens

[Docs: langchain.org/docs/modules/model_io/output_parsers]

StrOutputParser

The simplest parser: extracts string content from LLM output.

```
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-4o-mini")
chain = prompt | model | StrOutputParser()

result = chain.invoke({"text": "Hello, world!"})
print(result)  # "Bonjour, le monde!"
```

When to use: Raw string output for summarization, translation, open-ended generation.

CommaSeparatedListOutputParser

Parses comma-separated strings into a Python list.

```
from langchain_core.output_parsers import (
    CommaSeparatedListOutputParser)

parser = CommaSeparatedListOutputParser()
prompt = ChatPromptTemplate.from_template(
    "List 5 fruits:\n{format_instructions}")

chain = prompt | model | parser
result = chain.invoke({})
# result = ["mango", "pineapple", "banana", ...]
```

Use case: Simple list generation with delimiter-separated values.

JsonOutputParser

Parses JSON strings from LLM output.

```
from langchain_core.output_parsers import JsonOutputParser
from pydantic import BaseModel, Field

class PersonInfo(BaseModel):
    name: str = Field(description="Full name")
    age: int = Field(description="Age in years")

parser = JsonOutputParser(pydantic_object=PersonInfo)
prompt = ChatPromptTemplate.from_template(
    "Extract person info:\n{text}\n"
    "{format_instructions}")

chain = prompt | model | parser
```

[Docs: langchain.org/docs/modules/model_io/output_parsers/json]

PydanticOutputParser

Combines Pydantic schema validation with output parsing.

```
from langchain_core.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field

class Product(BaseModel):
    name: str = Field(..., description="Product name")
    price: float = Field(..., description="Price in USD")
    in_stock: bool = Field(default=True)

parser = PydanticOutputParser(pydantic_object=Product)
chain = prompt | model | parser
```

[Docs: langchain.org/docs/modules/model_io/output_parsers/pydantic]

The Modern Approach: with_structured_output()

ChatModels now support native structured output.

```
from pydantic import BaseModel, Field
from langchain_openai import ChatOpenAI

class Sentiment(BaseModel):
    sentiment: str = Field(description="positive|negative|neutral")
    score: float = Field(description="Confidence 0-1")

model = ChatOpenAI(model="gpt-4o-mini")
structured_model = model.with_structured_output(Sentiment)

result = structured_model.invoke(
    "I love this product!")
```

Advantages: No explicit parser, leverages model's native capabilities, better reliability

[Docs: langchain.org/docs/modules/model_io/chat/structured_output]

with_structured_output() Under the Hood

Two Implementation Strategies

1. Function Calling (Preferred):

Model uses function calling API to return structured data directly.

Pros: Most reliable. Cons: Requires function calling support.

2. JSON Mode:

Model constrained to output valid JSON matching schema.

Pros: Wider support. Cons: Additional prompt engineering.

[Docs: langchain.org/docs/concepts/structured_output]

Defining Schemas with Pydantic BaseModel

Pydantic models define the expected output structure.

```
from pydantic import BaseModel, Field
from typing import List, Optional

class Article(BaseModel):
    title: str = Field(..., description="Article headline")
    author: str = Field(..., description="Author name")
    tags: List[str] = Field(
        default_factory=list, description="Topic tags")
    summary: Optional[str] = Field(None)

model = ChatOpenAI(model="gpt-4o-mini")
structured_model = model.with_structured_output(Article)
```

[Docs: pydantic.dev]

Complex Nested Schemas

Output parsers handle deeply nested Pydantic structures.

```
class Address(BaseModel):
    street: str
    city: str
    country: str

class Person(BaseModel):
    name: str
    age: int
    address: Address
    contact_emails: List[str]

model = ChatOpenAI(model="gpt-4o-mini")
parser = model.with_structured_output(Person)
# Full nested object automatically parsed
```

Enum Outputs and Constrained Choices

Use enums to restrict output to specific predefined values.

```
from enum import Enum

class Priority(str, Enum):
    HIGH = "high"
    MEDIUM = "medium"
    LOW = "low"

class Task(BaseModel):
    title: str
    priority: Priority
    completed: bool = Field(default=False)

model = ChatOpenAI(model="gpt-4o-mini")
parser = model.with_structured_output(Task)
# result.priority is guaranteed to be HIGH, MEDIUM, or LOW
```

Benefit: Enums ensure the LLM selects from allowed choices.

Optional Fields and Default Values

Handle optional data and set sensible defaults.

```
from typing import Optional, List

class Review(BaseModel):
    product_name: str
    rating: int = Field(ge=1, le=5)
    text: Optional[str] = Field(None)
    tags: List[str] = Field(default_factory=list)
    verified_purchase: bool = Field(default=False)

model = ChatOpenAI(model="gpt-4o-mini")
parser = model.with_structured_output(Review)

# Only required fields must be present
result = parser.invoke("Product: Laptop, Rating: 5")
# rating=5, text=None, tags=[], verified_purchase=False
```

[Docs: pydantic.dev/docs/concepts/fields]

Custom Parsers with RunnableLambda

Build custom parsing logic using RunnableLambda.

```
from langchain_core.runnables import RunnableLambda

def parse_custom(text: str) -> dict:
    lines = text.strip().split('\n')
    result = {}
    for line in lines:
        if ':' in line:
            key, value = line.split(':', 1)
            result[key.strip()] = value.strip()
    return result

custom_parser = RunnableLambda(parse_custom)
chain = prompt | model | custom_parser
```

Use: Combine custom logic with LangChain for specialized parsing.

Composition: Chaining Parsers

Combine multiple parsers for complex workflows.

```
def validate_json(data: dict) -> dict:
    if data.get('age', 0) < 0:
        raise ValueError("Age cannot be negative")
    return data

json_parser = JsonOutputParser(pydantic_object=dict)
validator = RunnableLambda(validate_json)

# Chain: model to JSON parse to custom validation
chain = prompt | model | json_parser | validator
```

Error Handling: OutputParserException

Handle parsing failures gracefully.

```
from langchain_core.exceptions import OutputParserException
from pydantic import BaseModel

class Data(BaseModel):
    value: int

parser = PydanticOutputParser(pydantic_object=Data)

try:
    result = parser.parse("invalid json")
except OutputParserException as e:
    print(f"Parsing failed: {e}")
    # Handle: retry, use default, log, etc.
```

Common Causes:

- LLM returns malformed JSON
- Missing required fields
- Type mismatch (string vs int)
- Enum value not in allowed set

Retry Patterns

Automatically retry parsing with modified prompts.

```
class Task(BaseModel):
    title: str
    priority: str

parser = PydanticOutputParser(pydantic_object=Task)
prompt = ChatPromptTemplate.from_template(
    "Extract task:\n{text}\n{format_instructions}\n"
    "Return ONLY valid JSON, no extra text.")

chain = (prompt | model | parser).with_fallbacks(
    [prompt | model | parser])

result = chain.invoke({"text": "Important: fix bug"})
```

[Docs: langchain.org/docs/concepts/fallbacks]

Structured Retry Logic

Implement custom retry with exponential backoff.

```
import time
from langchain_core.exceptions import OutputParserException

def parse_with_retry(text, parser, max_retries=3):
    for attempt in range(max_retries):
        try:
            return parser.parse(text)
        except OutputParserException as e:
            if attempt == max_retries - 1:
                raise
            wait_time = 2 ** attempt
            time.sleep(wait_time)

result = parse_with_retry(llm_output, parser)
```

Comparison: Which Parser to Use?

Parser	Complexity	Reliability	Speed
StrOutputParser	Low	High	Very Fast
CommaSeparatedList	Low	Medium	Fast
JsonOutputParser	Medium	Medium	Medium
PydanticOutputParser	Medium	High	Medium
with_structured_output	Medium	Very High	Fast
Custom Lambda	High	Depends	Depends

Decision Guide:

- **Just strings?** Use StrOutputParser
- **Typed objects?** Use with_structured_output
- **Custom logic?** Use RunnableLambda

When to Use Function Calling vs JSON Mode

Function Calling
Best for: Maximum reliability, strict schema enforcement
When: Model supports it (GPT-4, Claude 3+)
Trade-off: Slightly higher latency

JSON Mode
Best for: Balance of reliability and speed
When: Model supports JSON mode but not function calling
Trade-off: Requires explicit prompting

Prompt plus Parsing
Best for: Flexibility, working with any model
When: Model has no structured output support
Trade-off: Less reliable, more manual validation needed

[Docs: langchain.org/docs/concepts/structured_output]

Parser Selection Guide

Scenario	Recommended Parser
Simple text output	StrOutputParser
List of items	CommaSeparatedListOutputParser
Typed, validated data	with_structured_output()
Custom extraction	RunnableLambda
Legacy Pydantic	PydanticOutputParser
Nested objects	with_structured_output() with nested Pydantic
Constrained choices	with_structured_output() with Enum

Performance Considerations

Function Calling is Faster

- No extra prompt engineering needed
- Single API call with structured result
- Model enforces schema natively
- 10-20 percent lower latency than JSON mode

Prompting Can Be Slow

- Extra tokens for format instructions
- May require multiple retries
- Additional parsing overhead
- Less reliable with smaller models

Optimization:

- Use with_structured_output when available
- Cache format instructions in prompt
- Batch similar parsing requests
- Consider model choice (GPT-4o better than GPT-3.5)

Token Usage and Cost

with_structured_output():

- Function calling: 50-100 extra tokens for schema
- JSON mode: 100-200 extra tokens for constraints
- Much cheaper than retry failures

Cost Comparison (for 1000 requests):

Approach	Relative Cost
with_structured_output()	1.0x
JSON Mode	1.1x
Prompting plus Parsing	1.3x (plus retries)

Recommendation: Use function calling when available. Cost difference is minimal but reliability gains are significant.

Quick Cheat Sheet

Import what you need:

```
from langchain_core.output_parsers import (
    StrOutputParser,
    JsonOutputParser,
    PydanticOutputParser,
    CommaSeparatedListOutputParser,
)
from langchain_core.runnables import RunnableLambda
from pydantic import BaseModel, Field
```

Define your schema:

```
class MyOutput(BaseModel):
    field1: str = Field(description="...")
    field2: int = Field(ge=0, le=100)

model.with_structured_output(MyOutput)
```

[Docs: langchain.org/docs/modules/model_io/output_parsers]

Summary: Best Practices

1. **Prefer with_structured_output()**. Modern, reliable approach for most use cases.
2. **Define clear Pydantic schemas**. Use Field descriptions and validators for clarity.
3. **Handle errors gracefully**. Catch OutputParserException and implement retries.
4. **Choose the right parser for the job**. Simple string: StrOutputParser. Typed data: with_structured_output.
5. **Test with your model**. Behavior varies by model.
6. **Monitor failure rates**. Track parsing exceptions and adjust prompts as needed.
7. **Validate output in application**. Don't trust parser alone.

Common Gotchas

JSON Format Not Guaranteed
LLMs may return malformed JSON even with instructions. Action: Always use error handling and retries.

Enum Values Must Match Exactly
If your Enum says HIGH, model must output HIGH. Action: Be explicit in prompts.

Optional Fields Can Be Tricky
Models might skip optional fields or include them as null. Action: Set explicit defaults in Pydantic.

Large Nested Schemas Are Expensive
Deep nesting increases token count significantly. Action: Flatten where possible, split into multiple requests.

Self-Check Questions

1. What is the difference between StrOutputParser and PydanticOutputParser?
2. When would you use with_structured_output() vs custom RunnableLambda?
3. How do Enum fields help ensure valid outputs?
4. What does OutputParserException tell you?
5. Is function calling or JSON mode more reliable? Why?
6. How would you handle a case where the LLM returns invalid JSON?
7. What is the advantage of setting Field descriptions in Pydantic models?
8. How would you validate that optional fields are handled correctly?

[Docs: langchain.org/docs/modules/model_io/output_parsers]

Resources and Further Reading

- **LangChain Docs: Output Parsers**
  langchain.org/docs/modules/model_io/output_parsers
- **Pydantic Documentation** docs.pydantic.dev
- **Structured Outputs (Concepts)**
  langchain.org/docs/concepts/structured_output
- **OpenAI Function Calling** platform.openai.com/docs/guides/function-calling
- **LangChain Source Code** github.com/langchain-ai/langchain

Next in this series: Section 03 - Prompt Engineering and Templates

Questions?

Output parsing bridges the gap between
unstructured LLM output and structured application logic.

LangChain v1.2.x — March 2026